VendEletro — Diagramas UML e Arquitetura gerados com LLM

PCA008 — Engenharia de Software (PPGCA/UPF, 2026.2) · LLM utilizada: Claude Opus 5.5 (Claude Code) · 05/11/2026

1. Método

O trabalho segue o artigo *Automating Software Diagram Generation with Large Language Models* (Gheorghita, Irimia e Iftene, Procedia Computer Science 270, 2025). Nele, a ferramenta DiagramAI pede a LLMs diagramas de casos de uso, classes, sequência, arquitetura e C4 em **código Mermaid**. A saída é avaliada por dois critérios: **corretude sintática** (o código renderiza?) e **consistência estrutural** (elementos e relações corretos para o tipo de diagrama). O artigo também usa **prompts de edição** para corrigir os diagramas.

Aplicação ao VendEletro:

1. Entrada: as 14 Histórias de Usuário e 7 Histórias Técnicas (INVEST) do VendEletro.
2. Um prompt por tipo de diagrama, com regras de sintaxe derivadas dos erros relatados no artigo.
3. Validação de cada diagrama com o Mermaid CLI (`mmdc`). Quando houve erro, ele foi devolvido à LLM num prompt de correção.
4. Avaliação com nota de 0 a 4 nos dois critérios.

O prompt original enviado à LLM está no Apêndice. Todos os arquivos `.mmd` e `.png` estão nas pastas `tarefa1/` e `tarefa2/`.

2. Tarefa 1 — Diagramas UML

2.1 Diagrama comportamental — Casos de uso

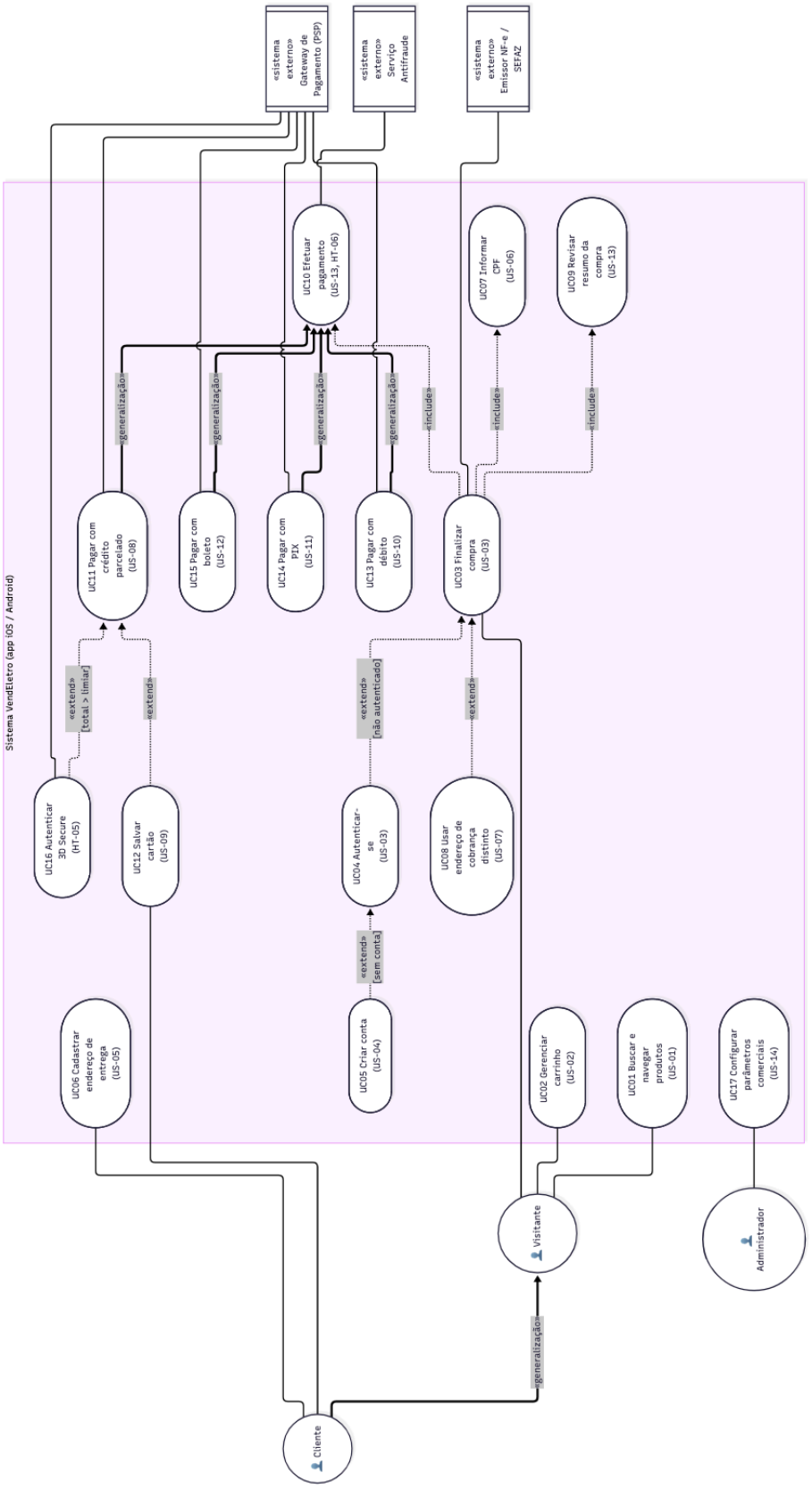


Figura 1 — Diagrama de casos de uso do VendEletro (girado 90° para caber na página)

2.2 Diagrama estrutural — Classes

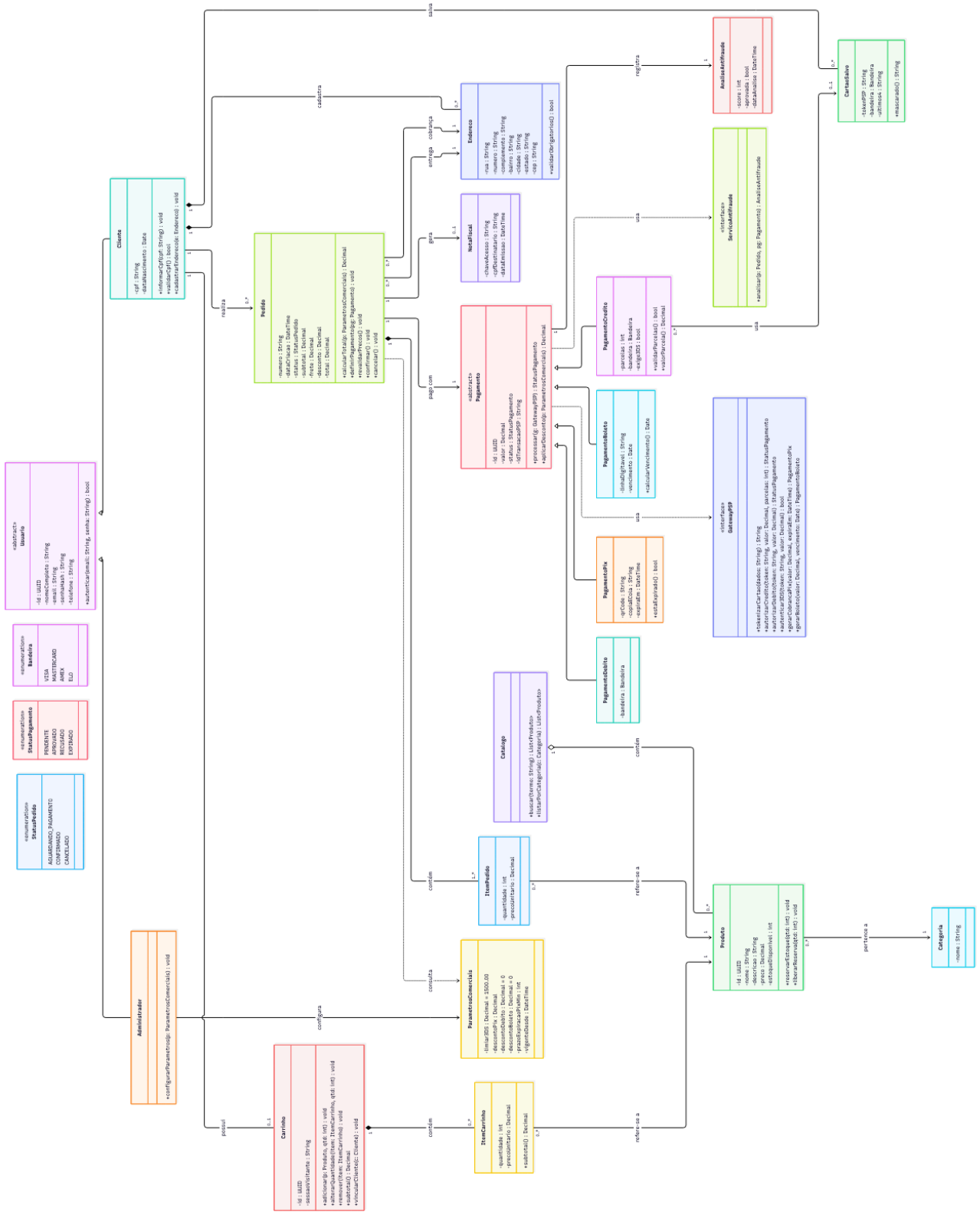


Figura 2 — Diagrama de classes do domínio VendEletro (girado 90° para caber na página)

2.3 Diagramas comportamentais — Sequência

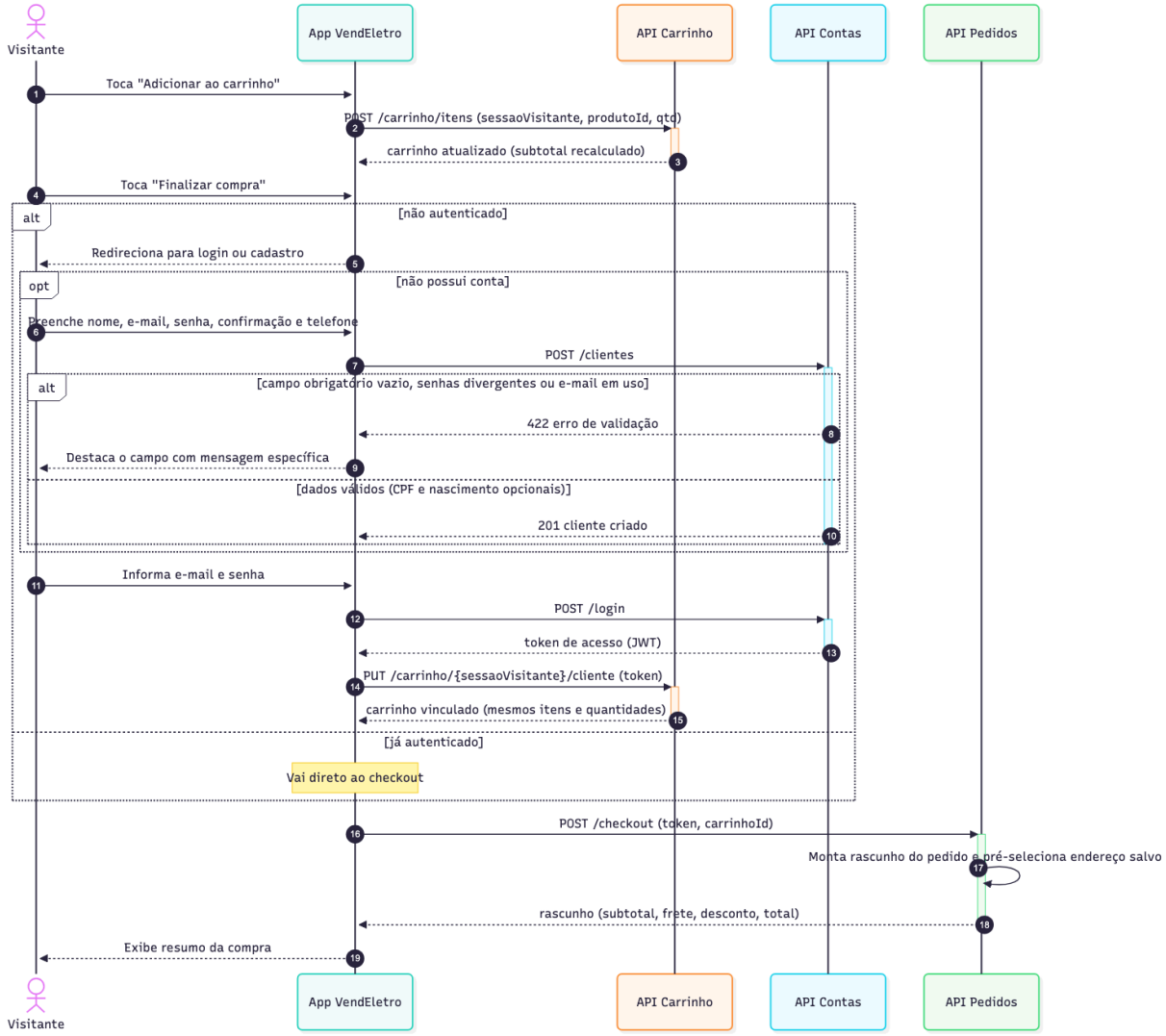


Figura 3 — Sequência 1: carrinho sem login e autenticação no checkout (US-02 a US-05, US-13)

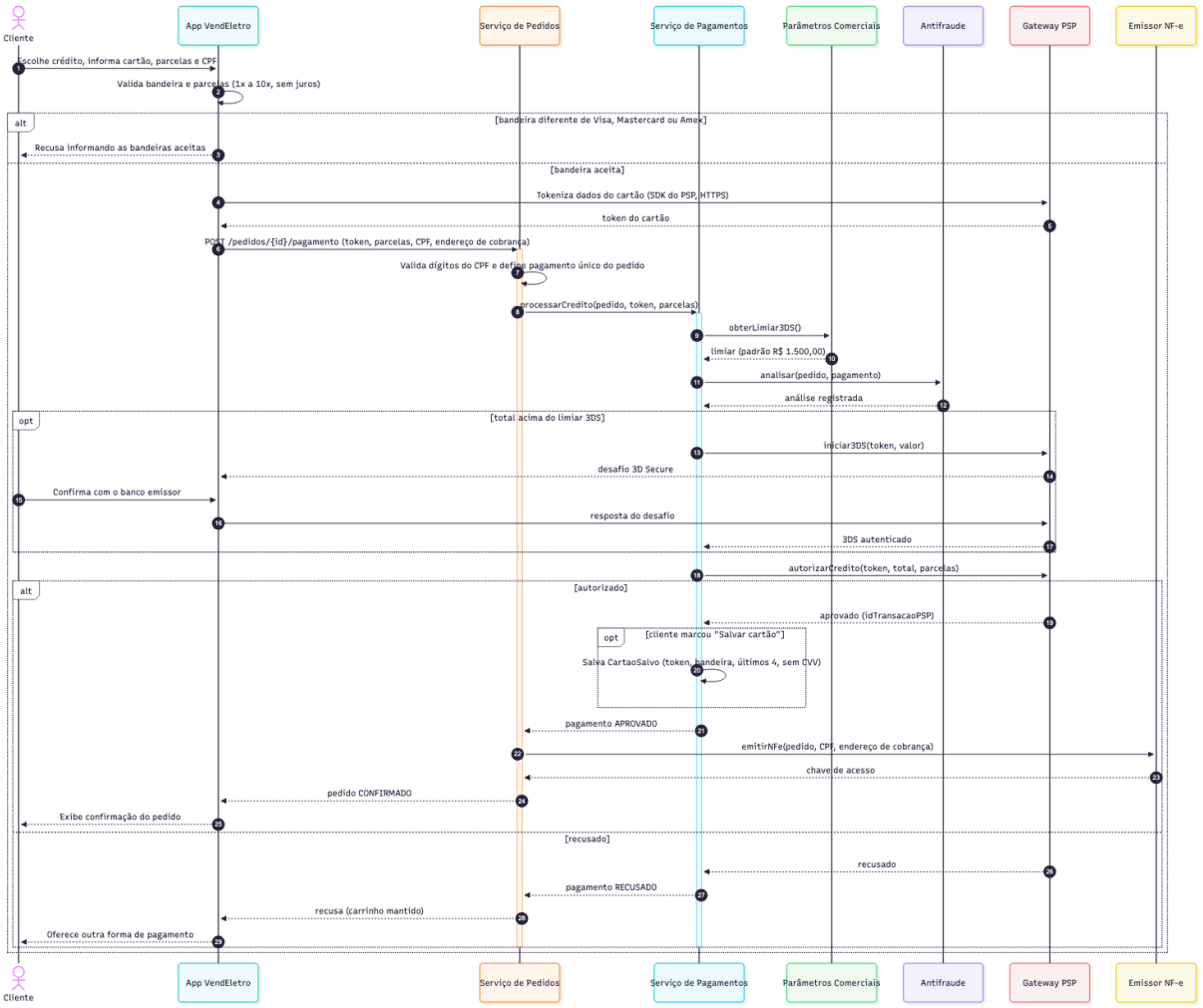


Figura 4 — Sequência 2: pagamento com crédito parcelado, antifraude e 3D Secure (US-06 a US-09, HT-04 a HT-07)

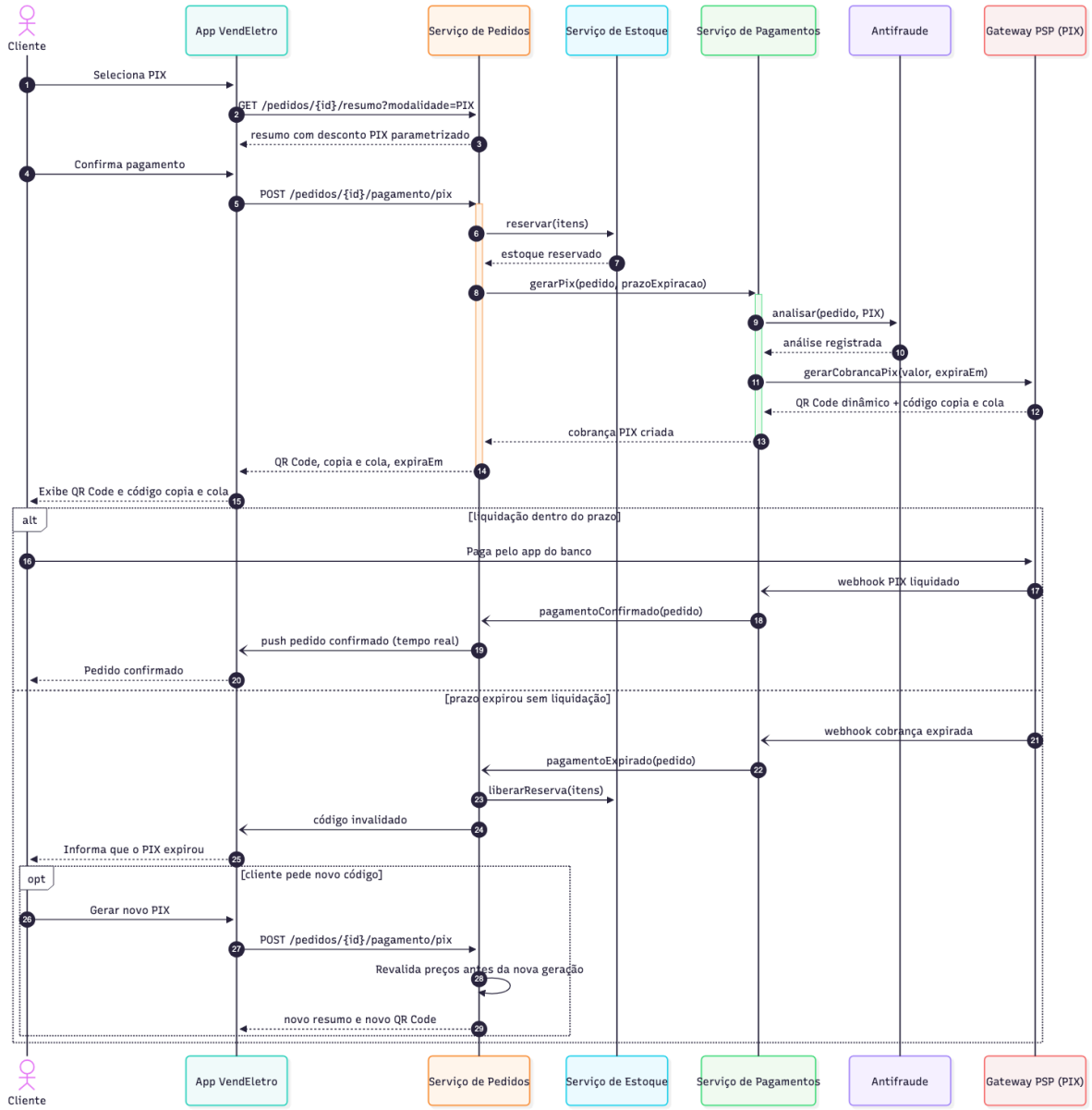


Figura 5 — Sequência 3: pagamento com PIX, liquidação e expiração (US-11, US-14)

2.4 Avaliação

Os critérios são os mesmos da seção 4.1 do artigo:

- **Corretude sintática**: o código Mermaid renderiza sem erro? Verificado com `mmdc`.
- **Consistência estrutural**: os elementos e relações exigidos pelo tipo de diagrama estão presentes e corretos? Verificado com checklist manual contra as histórias.

Notas de 0 a 4, como na Tabela 1 do artigo. **Atenção:** é uma autoavaliação (a mesma LLM gerou e avaliou) e deve ser lida com esse viés em mente.

Diagrama	Tentativas até renderizar	Erros encontrados	Sintaxe (0–4)	Estrutura (0–4)	Observações
Casos de uso	1	—	4,0	3,5	O Mermaid não tem notação de casos de uso; generalização e «include»/«extend» são aproximadas com setas rotuladas (mesma limitação da Fig. 3a do artigo). Todas as 14 US estão cobertas.
Classes	1	—	4,0	4,0	24 classes/interfaces/enums, todas declaradas; herança com a seta de generalização do Mermaid (sem uso de `:`); multiplicidades em todas as associações.
Sequência 1 — checkout/login	1	—	4,0	4,0	Cobre US-02 c.1–3, US-03 c.1–3, US-04 c.1/3/4, US-05 c.3 e US-13 c.1.
Sequência 2 — crédito	**3**	alias `Par` = palavra reservada `par`; `;` dentro da mensagem	2,0 → 4,0	4,0	Os dois erros são do tipo descrito no artigo ("unsupported symbols in names or descriptions"). Foram corrigidos com prompts de edição.
Sequência 3 — PIX	1	—	4,0	4,0	Cobre US-11 c.1–5, com mensagens assíncronas para os webhooks.
C4 Contexto (v1, `C4Context`)	1	rótulos sobrepostos	4,0	2,5	Sintaxe válida, mas layout ilegível. Reproduz o achado do artigo de que C4 gerado direto em Mermaid é o tipo mais frágil.
C4 Contexto (v2, `flowchart`)	1	—	4,0	4,0	Mesmos elementos, com a notação visual do C4.
Componentes	1	—	4,0	3,5	Completo, mas denso (muitas arestas para os bancos de dados).

Checklist de rastreabilidade (US → diagramas)

História	Casos de uso	Classes	Sequência
US-01 Buscar produtos	UC01	Catalogo, Produto, Categoria	—

US-02 Carrinho sem login	UC02	Carrinho, ItemCarrinho	Seq 1
US-03 Login no checkout	UC03, UC04	Carrinho.vincularCliente	Seq 1
US-04 Criar conta	UC05	Usuario, Cliente	Seq 1
US-05 Endereço de entrega	UC06	Endereco	Seq 1
US-06 CPF no checkout	UC07	Cliente.validarCpf, NotaFiscal	Seq 2
US-07 Endereço de cobrança	UC08	Pedido → Endereco (cobrança)	Seq 2
US-08 Crédito parcelado	UC11	PagamentoCredito, Bandeira	Seq 2
US-09 Salvar cartão	UC12	CartaoSalvo	Seq 2
US-10 Débito	UC13	PagamentoDebito	—
US-11 PIX	UC14	PagamentoPix	Seq 3
US-12 Boleto	UC15	PagamentoBoleto	—
US-13 Resumo	UC09	Pedido.calcularTotal	Seq 1, 2, 3
US-14 Parâmetros	UC17	ParametrosComerciais, Administrador	Seq 2, 3
HT-04 Tokenização	—	CartaoSalvo.tokenPSP, GatewayPSP	Seq 2
HT-05 3D Secure	UC16	PagamentoCredito.exige3DS	Seq 2
HT-06 Antifraude	UC10 ↔ Antifraude	ServicoAntifraude, AnaliseAntifraude	Seq 2, 3
HT-07 PSP	sistema externo PSP	GatewayPSP	Seq 2, 3

Débito (US-10) e boleto (US-12) não têm diagrama de sequência próprio. O fluxo deles é o mesmo do crédito, sem parcelas e sem 3DS, e no caso do boleto com confirmação assíncrona igual à do PIX.

3. Tarefa 2 — Arquitetura

3.1 Estilo escolhido

Cliente-Servidor + Monólito Modular em Camadas, com integração de pagamentos Orientada a Eventos; MVC/MVVM na apresentação.

Estilo	Papel na solução	Motivo (histórias / RNF)
Cliente-Servidor	App iOS/Android e painel admin (clientes) ↔ backend (servidor)	RNF-UNI-02 exige app nas duas lojas; preço, estoque, descontos e limiar 3DS precisam ficar no servidor para mudar "sem nova versão do app" (US-14).
Camadas	Apresentação → API/BFF → Aplicação (módulos de domínio) → Dados	Isola as regras RN-UNI-01..09 da UI e da persistência; cada camada pode ser testada isoladamente (critério T do INVEST).
Monólito modular	Um deployable, com módulos alinhados aos épicos (Catálogo, Carrinho, Contas, Pedidos, Pagamentos, Parâmetros, Fiscal, Estoque)	Backlog de 89 pontos e equipe de MVP; o checkout (pedido + reserva + pagamento) fica numa transação local, sem saga distribuída. As fronteiras entre módulos permitem extrair serviços depois.
Orientada a Eventos	Barramento interno (outbox + fila) para `PagamentoAprovado`, `PagamentoRecusado`, `PixExpirado`, `BoletoVencido`	PIX e boleto se confirmam de forma assíncrona por webhook do PSP (US-11, US-12); a confirmação em tempo real (RNF-UNI-08), a emissão de NF-e, a liberação de estoque e o push reagem ao mesmo evento sem acoplar o módulo de Pagamentos a cada um deles.
MVC / MVVM	Dentro do app (MVVM) e do painel web (MVC)	Separa telas de estado e regras de UI; facilita o fluxo "voltar ao checkout com o mesmo carrinho" (US-03).

3.2 Estilos descartados como estilo principal

- **Monolítico não modular**: misturaria o código sensível de pagamento (escopo PCI) com o catálogo e dificultaria a evolução.
- **Microsserviços**: o custo operacional (deploy independente, observabilidade, transações distribuídas) é desproporcional a um MVP de 89 pontos. Fica como caminho de evolução, começando por extrair Pagamentos.
- **SOA**: um ESB e contratos corporativos pesados não se justificam, já que não há vários sistemas legados para orquestrar.
- **Pipes-and-Filters**: serve para processamento em lote ou em fluxo; o VendEletro é interativo. Pode aparecer de forma pontual, por exemplo num pipeline de regras antifraude.
- **Peer-to-Peer**: não há pares equivalentes; um e-commerce precisa de uma autoridade central sobre preço, estoque e pagamento.

3.3 RNF → decisão arquitetural

RNF	Decisão
RNF-UNI-01 Busca ≤ 2,0 s	Índice de busca dedicado (OpenSearch) + cache Redis; o módulo Catálogo lê do índice, não do banco transacional.

RNF-UNI-02 iOS e Android	Cliente mobile único (nativo ou multiplataforma) falando com uma API REST/BFF.
RNF-UNI-03 HTTPS	TLS terminado no API/BFF; requisições HTTP sem criptografia são recusadas.
RNF-UNI-04 Tokenização	O app envia o cartão direto ao SDK do PSP; o backend só recebe e guarda o token (fora do escopo PCI completo).
RNF-UNI-05 3D Secure	O módulo Pagamentos lê `limiar3DS` de Parâmetros Comerciais (em cache) antes de autorizar.
RNF-UNI-06 Antifraude 100%	Pagamentos chama Antifraude antes de qualquer autorização e persiste `AnaliseAntifraude`.
RNF-UNI-07 PSP reconhecido	Adaptador `GatewayPSP` (portas e adaptadores) permite trocar Cielo, PagSeguro ou Mercado Pago, em homologação e produção.
RNF-UNI-08 Tempo real	Webhooks do PSP → evento no barramento → Pedidos confirma → push (APNs/FCM) para o app.

3.4 Diagramas de arquitetura

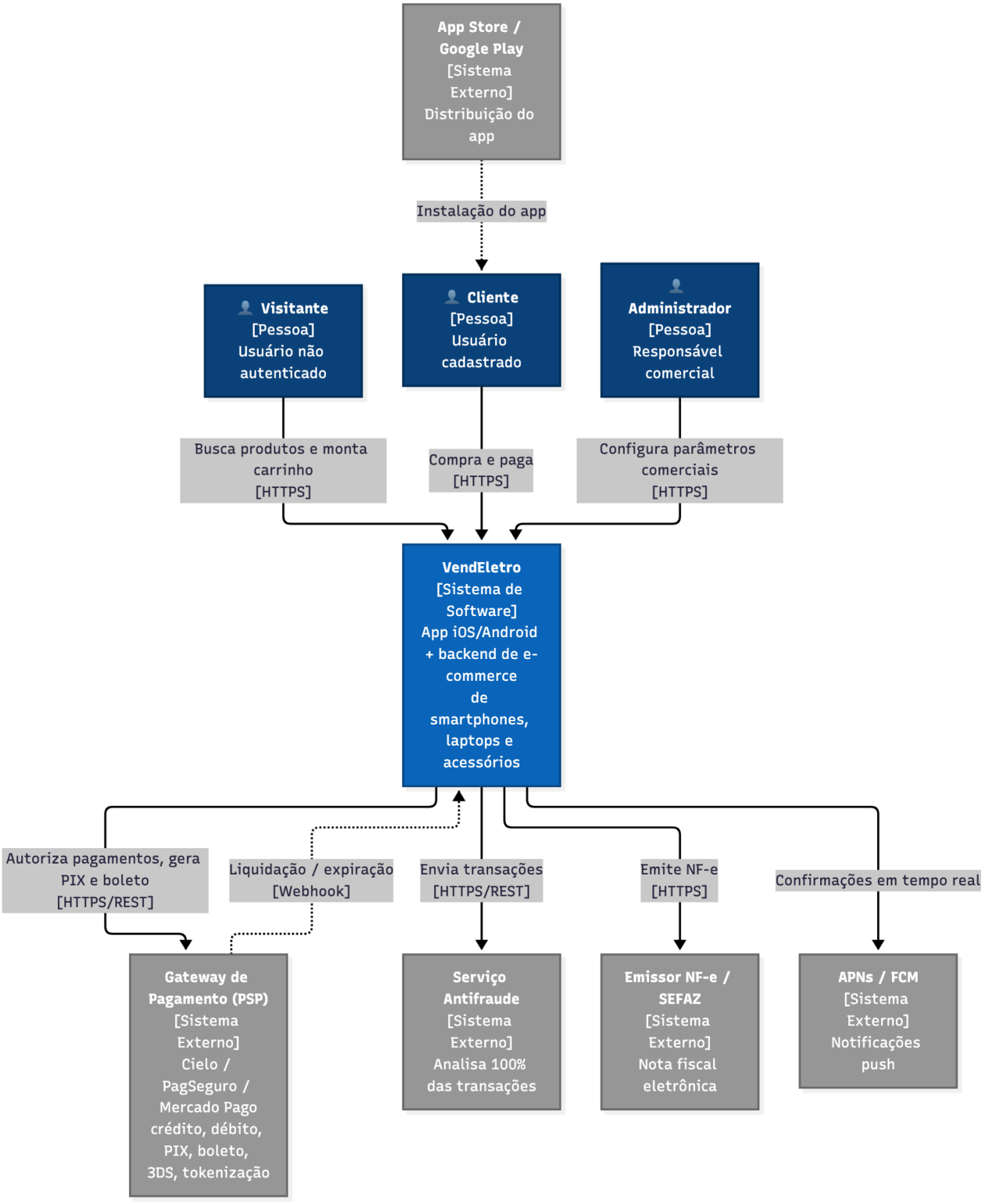


Figura 6 — Contexto C4 (nível 1), versão final em flowchart

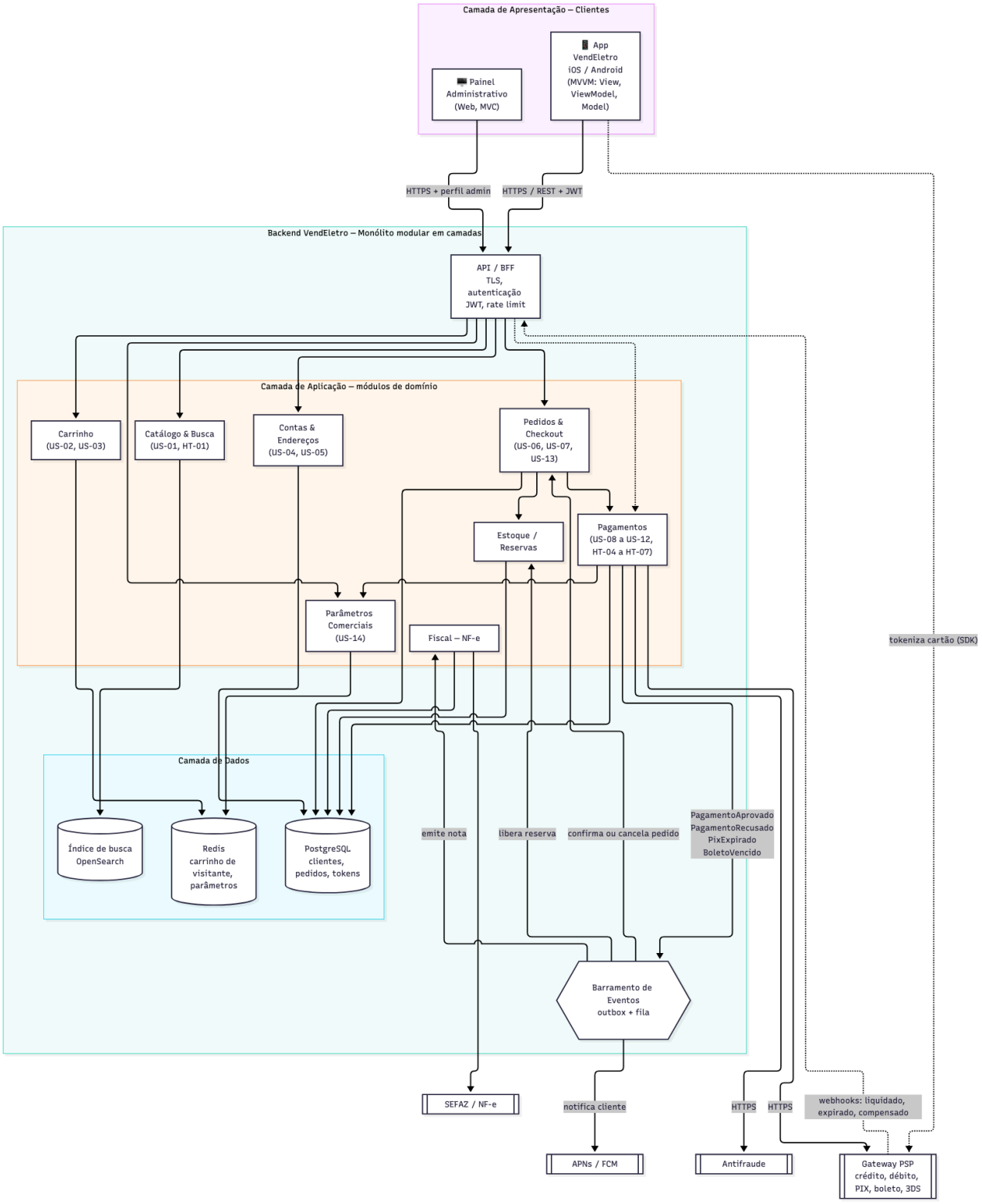


Figura 7 — Componentes: Cliente-Servidor + monólito modular em camadas + eventos

> **Nota sobre a Figura 6:** a primeira versão do diagrama de contexto foi gerada com a sintaxe `C4Context` nativa do Mermaid. O código era válido, mas o layout automático sobrepôs os rótulos das setas e o diagrama ficou ilegível. Por isso ele foi refeito como `flowchart` com a notação visual do C4. Isso confirma o achado do artigo de que diagramas C4 gerados diretamente em Mermaid são os mais frágeis. A versão descartada está em `tarefa2/arquitetura-c4-contexto-v1-c4nativo.png`.

3.5 Riscos e evolução

- **Ponto único de falha**: o monólito é escalado horizontalmente (várias instâncias sem estado atrás de um balanceador); o carrinho do visitante fica no Redis.
- **Evento perdido entre o banco e a fila**: padrão Transactional Outbox (o evento é gravado na mesma transação do pedido).
- **Webhook duplicado do PSP**: os consumidores são idempotentes, usando `idTransacaoPSP` como chave.
- **Crescimento**: se Pagamentos ou Catálogo exigirem escala ou ritmo de deploy próprios, extrair esses módulos como microsserviços. O barramento de eventos já existente reduz o custo dessa migração.

4. Consumo de tokens

4.1 Estimativa por mensagem (texto visível do chat)

Estimativa: **tokens ≈ caracteres ÷ 4**. É a aproximação usual para tokenizadores BPE; em português e em código Mermaid o número real tende a ser de 10% a 30% maior.

Tarefa	Mensagem	Tipo	Caracteres	Tokens (est.)
Tarefa 1	Pedido original (enunciado)	entrada (prompt)	914	229
Tarefa 1	Prompt 1 — Casos de uso	entrada (prompt)	1.280	320
Tarefa 1	Resposta 1	saída (resposta)	2.848	712
Tarefa 1	Prompt 2 — Classes	entrada (prompt)	1.225	307
Tarefa 1	Resposta 2	saída (resposta)	6.379	1.595
Tarefa 1	Prompt 3 — Sequência	entrada (prompt)	1.209	303
Tarefa 1	Resposta 3	saída (resposta)	6.688	1.672
Tarefa 1	Prompt 4 — Correção (erro de sintaxe)	entrada (prompt)	186	47
Tarefa 1	Resposta 4	saída (resposta)	480	120
Tarefa 1	Prompt 5 — Correção (erro de sintaxe)	entrada (prompt)	64	16
Tarefa 1	Resposta 5	saída (resposta)	2.662	666
Tarefa 2	Prompt 1 — Arquitetura	entrada (prompt)	1.272	318
Tarefa 2	Resposta 1	saída (resposta)	7.950	1.988
Tarefa 2	Prompt 2 — Correção (legibilidade do C4)	entrada (prompt)	310	78
Tarefa 2	Resposta 2	saída (resposta)	2.158	540

Resumo	Entrada (est.)	Saída (est.)	Total (est.)
Tarefa 1	1.222	4.765	5.987
Tarefa 2	396	2.528	2.924
Total	**1.618**	**7.293**	**8.911**

4.2 Contexto anexado (não aparece no texto do chat)

Os PDFs foram enviados à LLM como imagens de página. Cada página consome em torno de 1.500 a 1.600 tokens de entrada:

Anexo	Páginas	Tokens (est.)
Artigo (Gheorghita et al., 2025)	10 lidas de 13 (as 3 últimas são conclusão e referências)	≈ 15.500
Histórias de Usuário VendEletro	13	≈ 20.000
Total de anexos	23	**≈ 35.500**

4.3 Consumo real da sessão (Claude Code)

O Claude Code reenvia todo o histórico a cada chamada: anexos, chamadas de ferramenta (leitura de PDFs, renderização com `mmdc`, geração do PDF) e o prompt de sistema. A maior parte disso vem do cache de prompt. Por isso o total real é bem maior que a soma das mensagens acima. Valores medidos pelo comando `/usage` do Claude Code ao final do trabalho (modelo `claude-opus-5-5`):

Métrica	Valor
Tokens de entrada (sem cache)	696
Tokens de saída	42.300
Tokens lidos do cache	5.700.000
Tokens gravados no cache	160.100
Requisições à API	44 (97% da entrada servida pelo cache)
Custo total	US$ 3,26
Duração (tempo de API)	11 min 33 s
Duração (tempo de relógio)	56 min 54 s
Linhas geradas (arquivos)	1.062

Leitura dos números:

- Os 42,3 mil tokens de saída incluem, além das respostas do chat (≈ 7,3 mil), o código dos arquivos `.mmd`, o script que monta o relatório, as chamadas de ferramenta e o raciocínio interno do modelo.
- Os 5,7 milhões de tokens lidos do cache são o histórico (com os 2 PDFs) reenviado a cada uma das 44 requisições. Como o cache é cobrado a cerca de 10% do preço da entrada normal, o custo total ficou em US$ 3,26.
- Os valores foram medidos logo antes do turno final, que só preencheu esta tabela, então o total final é um pouco maior.

Apêndice — Prompt original

> Tarefas:
>
> 1 - Ler o artigo "Automating Software Diagram Generation with Large Language Models", e selecionar o sistema VendEletro e a partir das User Stories gerar diagramas da UML. Diagramas Estruturais (Estáticos -> diagrama de classe) e Diagramas Comportamentais (Dinâmicos -> diagrama de casos de uso e diagrama de sequência).
>
> 2. Solicite a uma LLM a geração de uma arquitetura para o exercício VendEletro. São exemplos de padrões (estilos) arquiteturais:
>
> Monolítica, Camadas (Layers), Cliente-Servidor (Client-Server), MVC (Model-View-Controller), Microsserviços (Microservices), Orientada a Serviços (SOA), Orientada a Eventos (Event-Driven), Pipes-and-Filters, Peer-to-Peer (P2P):
>
> Em ambas tarefas, entregar o "chat" e o consumo de tokens (se pertinente)
>
> [Anexos: "Automating Software Diagram Generation with Large Language Models.pdf" (13 p.) e "Histórias de Usuário (INVEST) - VendEletro.pdf" (13 p.)]